PYTHON — PROJET DE FIN DE MODULE

CampusCafé

Programmer la caisse de la cafétéria du campus — sans IA générative

Public	Mastère 1 — travail **individuel**
Durée	4 semaines, du lancement à la remise du code (tag `v1.0`)
Évaluation	Soutenance individuelle de 20 min : démonstration, explication du code, modification en direct
Prérequis	Notebooks 01 à 04 et POO **parties 1 et 2** : classes, `__init__`, `self`, méthodes, attributs de classe, `__str__` / `__repr__`, `__eq__` / `__lt__`, `__len__` / `__contains__` / `__iter__`
Intervenante	Nesrine AZOUZ ABDELADHIM — 2026-2027

1. Le contexte

L'association des étudiants reprend la **cafétéria du campus**. Aujourd'hui, tout se fait sur un cahier : les badges sont rechargés « à la confiance », les ruptures de stock sont découvertes devant le client, et personne ne sait dire le soir combien la journée a rapporté.

Vous écrivez le cœur du futur logiciel de caisse : un programme Python, organisé en **quatre classes**, qui connaît la carte et les clients, accepte ou refuse les commandes, applique remises et fidélité, rejoue une journée de service et produit le rapport de clôture.

> **Ce que le projet évalue**
>
> Votre capacité à **écrire vous-même** des classes simples et propres, puis à **les expliquer et les faire évoluer seul**, devant l'enseignante. Un programme qui tourne mais que vous ne savez pas expliquer ne rapporte presque rien (section 8).

2. Les données

2.1 La carte

Prix TTC. Tous les produits ont un stock. Les **plats** ne sont servis qu'entre **11:30 et 14:00** (début inclus, fin exclue). Les heures sont des chaînes `"HH:MM"` : elles se comparent directement (`"11:30" <= heure < "14:00"`).

Code	Produit	Catégorie	Prix	Stock
B01	Café expresso	boisson chaude	1,20 €	100
B02	Thé	boisson chaude	1,30 €	60
B03	Chocolat chaud	boisson chaude	1,60 €	40

Code	Produit	Catégorie	Prix	Stock
B04	Jus d'orange	boisson froide	2,00 €	24
B05	Eau 50 cl	boisson froide	1,00 €	40
V01	Croissant	viennoiserie	1,20 €	30
P01	Sandwich jambon-beurre	plat	4,00 €	15
P02	Wrap végétarien	plat	4,60 €	10
P03	Plat du jour	plat	7,20 €	20
D01	Cookie	dessert	1,40 €	25
D02	Fruit de saison	dessert	0,80 €	30

2.2 Les clients

Profil	Paiement	Règle
étudiant	badge	le solde ne peut pas devenir négatif
alternant	badge	le solde ne peut pas devenir négatif
personnel	badge	découvert autorisé jusqu'à −10,00 €
visiteur	comptant	pas de badge : ni recharge, ni fidélité

Recharge : entre 5,00 € et 100,00 €, et le solde après recharge ne doit pas dépasser 150,00 €. Sinon, la recharge est refusée.

Au début de la journée : **Alice** (étudiante, 12,00 €), **Bruno** (alternant, 3,50 €), **Chloé** (étudiante, 25,00 €), **Farid** (personnel, 0,00 €).

3. Le traitement d'une commande

Une commande est passée par un client, à une heure donnée, et contient des lignes « produit × quantité ». Elle est traitée **dans cet ordre** :

1. **Vérifier** que chaque code produit existe, que chaque quantité est positive, que chaque produit est disponible (stock suffisant, créneau ouvert pour les plats).
2. **Calculer** le total, appliquer la **remise** de votre variante (axe A) puis l'**avantage fidélité** (axe B). Arrondir au centime.
3. **Encaisser** : vérifier que le badge peut payer (selon le profil), puis le débiter. Un visiteur paie comptant.
4. **Enregistrer** : retirer les quantités du stock, mettre à jour la fidélité, ranger la commande dans le journal.

> **Règle d'or : tout ou rien**
>
> Si une vérification échoue, la commande est **refusée** et **rien n'est modifié** : ni stock, ni solde, ni fidélité, ni journal. Le refus est compté avec son **motif** : `produit inconnu`, `quantité invalide`, `rupture de stock`, `hors créneau`, `solde insuffisant`.

> Comme dans l'exercice `vendre` du cours, une méthode qui peut échouer **renvoie** son résultat (par exemple le triplet `(acceptée, motif, montant)`) au lieu d'afficher un message.

4. Votre variante

Le jour du lancement, chacun tire au sort une combinaison parmi **9** (une règle par axe). Votre programme n'implémente **que** votre variante ; notez-la en tête du README.

> Ma variante : axe A : …… axe B : ……

	Axe A — Remise		Axe B — Fidélité (titulaires d'un badge)
A1	**Tarif solidaire** : étudiants −10 %, alternants −5 % sur le total ; visiteurs +0,50 € de frais de service.	**B1**	**Points** : 1 point par euro entier payé. À 30 points, la commande suivante reçoit −2,00 € et 30 points sont retirés.
A2	**Happy hour** : de 15:00 à 17:00 (fin exclue), −20 % sur les boissons pour les titulaires d'un badge ; le personnel a −5 % sur le total à toute heure.	**B2**	**Tampons** : chaque boisson chaude achetée donne un tampon ; la 8ᵉ est offerte, puis le compteur repart à zéro.
A3	**Gros panier** : si le total atteint 8,00 €, −10 % pour les étudiants et alternants, −5 % pour les autres.	**B3**	**Client fidèle** : dès que le client a cumulé 40,00 € d'achats, il obtient −5 % sur toutes ses commandes suivantes.

5. Ce qui est attendu

5.1 Les quatre classes

L'architecture est imposée ; les noms de méthodes, les paramètres et les détails sont libres.

Classe	Ce qu'elle contient	Ce qu'elle sait faire
`Produit`	code, nom, catégorie, prix, stock	dire s'il est disponible pour une quantité à une heure donnée ; se vendre ; se réapprovisionner ; se présenter ; se comparer (égalité par code, ordre par prix)
`Client`	nom, profil, solde, état de fidélité	se recharger (en refusant une recharge non conforme) ; dire s'il peut payer un montant ; être débité ; calculer son avantage fidélité
`Commande`	numéro (attribué automatiquement), client, heure, lignes	ajouter une ligne ; calculer son total ; `len(commande)` = nombre d'articles ; `produit in commande` ; `for ligne in commande` ; s'afficher comme un ticket

Classe	Ce qu'elle contient	Ce qu'elle sait faire
`Cafeteria`	la carte, les clients, le journal des commandes, le compte des refus par motif	traiter une commande (section 3) ; recharger un badge ; réapprovisionner ; produire le rapport de clôture

5.2 Les exigences techniques

- ☐ chaque classe a un `__init__` et un `__repr__` ; `__str__` là où l'affichage pour l'utilisateur diffère ;
- ☐ `Produit` définit `__eq__` et `__lt__`, et ils **servent** (recherche avec `in`, tri avec `sorted`, `max`) ;
- ☐ `Commande` définit `__len__`, `__contains__` et `__iter__` ;
- ☐ les constantes (plafond de recharge, découvert, taux de remise…) sont des **attributs de classe** ; le numéro de commande vient d'un **compteur de classe** ;
- ☐ aucune valeur par défaut mutable ; une liste reçue en paramètre est **copiée** si l'objet la conserve ;
- ☐ les méthodes de calcul **renvoient** leurs résultats : seules les méthodes d'affichage utilisent `print` ;
- ☐ au moins **20 assertions** de test, dont les cas limites : 11:30 et 14:00, solde exactement suffisant, découvert à −10,00 €, recharge de 100,00 €, stock exactement suffisant.

Périmètre : Python 3.10 ou plus, bibliothèque standard uniquement. Les notions des parties 1 et 2 du cours suffisent. Toute notion non vue (héritage, `@property`, `dataclass`, exceptions personnalisées…) est autorisée **à condition de savoir l'expliquer** en soutenance.

5.3 Le rapport de clôture

- le nombre de commandes acceptées, et le nombre de refus par motif ;
- le chiffre d'affaires et le total des remises et avantages accordés ;
- les 3 produits les plus vendus (en quantité) ;
- la carte triée par prix croissant avec le stock restant, en signalant les produits épuisés ;
- les badges dont le solde est inférieur à 3,00 €.

Bonus, une fois tout le reste terminé : un menu console (`input`) pour saisir des commandes à la main.

6. Le scénario de la journée

N°	Heure	Événement	Détail
1	07:30	Ouverture	—
2	07:45	Commande — Alice	Café expresso, Croissant
3	08:10	Commande — Farid	Café expresso × 2
4	08:30	Commande — Bruno	Chocolat chaud, Croissant × 3
5	08:35	Recharge — Bruno	20,00 €
6	08:40	Commande — Bruno	Chocolat chaud, Croissant × 3
7	09:15	Commande — visiteur	Thé, Cookie

N°	Heure	Événement	Détail
8	11:00	Commande — Chloé	Plat du jour
9	12:05	Commande — Chloé	Plat du jour, Jus d'orange, Cookie
10	12:10	Recharge — Alice	150,00 €
11	12:20	Commande — Alice	Wrap végétarien, Eau 50 cl, Fruit de saison, Café expresso
12	12:45	Commande — visiteur	Sandwich jambon-beurre × 16
13	13:00	Commande — Bruno	Sandwich jambon-beurre, Smoothie mangue
14	15:30	Commande — Alice	Café expresso, Cookie
15	16:00	Réapprovisionnement	Sandwich jambon-beurre + 10
16	17:30	Commande — Farid	Thé × 2
17	18:30	Clôture	Rapport de clôture

> **Calcul à la main, obligatoire**
>
> **Avant** de coder les commandes, calculez **sur papier** les événements **2, 6 et 11** pour votre variante : montant payé, solde du badge, état de la fidélité. Déposez la photo dans `conception/` et comparez avec votre programme. En cas d'écart, expliquez dans le journal de bord qui avait raison.

7. Organisation et règles du jeu

Sans IA générative

Ce projet sert à construire **votre** raisonnement : découper un problème, se tromper, comprendre, corriger. La soutenance évalue ce que vous savez faire **seul**.

Autorisé	Interdit
• la documentation officielle de Python • les notebooks et corrections du cours • discuter des idées **à l'oral** avec d'autres étudiants • poser des questions à l'enseignante	• tout outil d'IA générative (ChatGPT, Claude, Gemini, Copilot, Cursor, l'assistant de Colab…) pour écrire, compléter ou corriger du code • l'autocomplétion par IA de l'éditeur (à désactiver) • échanger ou recopier du code

Traces demandées : un dépôt Git privé partagé avec l'enseignante dès la semaine 1, avec des commits réguliers ; un `JOURNAL.md` court (date, ce que j'ai fait, ce qui a bloqué, comment je m'en suis sorti) ; la conception papier ; la charte signée.

Calendrier

Semaine	Travail	Jalon
S1	Lire, concevoir sur papier.	**J1** — schéma des 4 classes **à la main** (attributs + méthodes), pseudo-code du traitement d'une commande, 10 cas de test rédigés en français, calculs à la main.
S2	`Produit`, `Client` et leurs tests.	**J2** — point d'étape de 10 min : démonstration et questions sur le code.
S3	`Commande`, `Cafeteria`, variante, scénario.	Le scénario s'exécute jusqu'à la clôture.
S4	Rapport, tests, README, relecture.	**Remise** — tag `v1.0` **48 h avant** la soutenance.

Livrables : le code et les tests ; `README.md` (variante, lancement, choix) ; `conception/` ; `JOURNAL.md` ; `sortie_scenario.txt`.

8. La soutenance individuelle (20 min)

Sur **votre** ordinateur, écran partagé, outils d'IA désactivés, à partir du tag `v1.0`.

Temps	Phase	Ce qui vous sera demandé
4 min	Démonstration	Le scénario de la journée, puis un **scénario inconnu** de quelques événements fourni le jour même.
7 min	Explication	Expliquer ligne par ligne des méthodes choisies par l'enseignante ; **prédire** le résultat d'un appel avant de l'exécuter.
7 min	Modification en direct	Une petite évolution tirée au sort, réalisée seul en expliquant votre démarche.
2 min	Questions	Questions de cours liées à votre code.

Grille (sur 20)

Critère	Ce qui est observé	Points
Fonctionnement	Scénario complet, variante respectée, scénario inconnu traité sans plantage.	4
Conception et qualité	Responsabilités bien réparties entre les classes, exigences de la section 5.2, code lisible, tests pertinents.	4
Maîtrise et explication	Explication exacte de son propre code, prédictions justes, vocabulaire précis (classe, instance, attribut, méthode, `self`).	6
Modification en direct	Lire, localiser, modifier, tester — en autonomie.	5
Démarche	Jalons rendus, commits réguliers, journal honnête.	1

> **Clause de maîtrise**
>
> Une partie du code que vous ne savez pas expliquer est considérée comme **non réalisée**. Mieux vaut un programme plus modeste que vous maîtrisez entièrement qu'un programme complet dont des pans vous échappent.